

گزارش طراحی توابع `features_extra` و `shaping` در مدل جنگ فرسایشی ناقص

خلاصه اجرایی: در این پروژه، هدف پیاده‌سازی دو تابع `features_extra` و `shaping` در فایل `theory.py` بر اساس مدل جنگ فرسایشی چندمرحله‌ای با اطلاعات ناقص است. این گزارش شامل نگاشت دقیق متغیرهای کد به نمادهای نظری مدل، ارائه الگوریتم‌های احتمالی (پژوهش‌نما) برای تولید ویژگی‌ها و تنظیم پاداش، انتخاب پارامترها، نمونه‌های آزمون و شبیه‌سازی، تحلیل پایداری و حساسیت، و در نهایت قطعه‌های کد پیشنهادی و چک‌لیستی برای ادغام نهایی است. تمامی پیشنهادها بر مبنای مفاهیم نظری مدل (سیگنال‌دهی پرمزینه، تاب‌آوری متغیر، به‌روزرسانی باور بیزی، شرط تک‌عبوری، هزینه مؤثر و ارزش خروج وابسته به تاریخچه) طراحی شده‌اند.

۱. نگاشت متغیرهای کد به نمادهای نظری

در کد محیط شبیه‌سازی، فرض بر این است که هر مرحله جاری دارای وضعیت (تاریخچه عمومی) `h^t` و متغیرهای خصوصی و عمومی بازی است. جدول زیر ورودی‌ها/خروجی‌های کلیدی توابع را به نمادهای مدل مرتبط می‌کند:

متغیر/پارامتر در <code>theory.py</code>	نماد نظری مدل	توضیح
<code>state['endurance'][i]</code>	e_i^t	تاب‌آوری جاری بازیکن i در مرحله t
<code>state['capacity'][i]</code>	m_i یا $\bar{m}_i$	ظرفیت مدیریت هزینه اولیه (مقدار پایه) یا جاری بازیکن i
<code>state['raw_cost'][i]</code>	$c_i(t, h^t)$	فشار خام مادی/اقتصادی وارد بر بازیکن i در مرحله t
<code>state['audience_cost'][i]</code>	$a_i^{aud}(t, h^t)$	هزینه سیاسی/اجتماعی داخلی (فشار مخاطب) بر بازیکن i در مرحله t
<code>state['risk_cost'][i]</code>	$r_i(t, h^t)$	هزینه خطر تشدید یا فروپاشی (ریسک) بر بازیکن i در مرحله t
<code>state['signal'][i]</code>	σ_i^t	قدرت یا نوع سیگنال ارسال‌شده توسط بازیکن i در مرحله t (معادل C یا E)
خروجی تابع	$\phi_i(\sigma_i^t, \theta_i, e_i^t, h^t)$	هزینه سیگنال‌دهی بازیکن (ممکن است ترکیبی از تولید و تعهد)
<code>state['exit_value'][i]</code>	$X_i(t, h^t)$	ارزش خروج بازیکن i در تاریخچه h^t

برون‌داد `shaping` پاداش تعدیل شده پاداش نهایی که پس از اضافه‌کردن تقویت (shaping) به عامل RL داده می‌شود

توضیح: بسیاری از این متغیرها در کلاس محیط بازی تعریف شده و به‌صورت دیکشنری یا آرایه در دسترس هستند. به‌عنوان مثال، اگر در کد از `state['endurance']` برای نگهداشتن مقادیر تاب‌آوری استفاده شده باشد، آن مقادیر دقیقاً معادل e_i^t در نظریه هستند. مشاهده می‌شود که هزینه مؤثر g_i از ترکیب سه فشار خام c_i, a_i^{aud}, r_i و ظرفیت m_i محاسبه می‌شود (معادله

$ffeg : qe$

در مدل). توابع `features_extra` و `shaping` می‌توانند مستقیماً از این اجزا استفاده کنند یا آن‌ها را برای فرمول‌های بعدی ترکیب نمایند.

۲. الگوریتم پیشنهادی برای `features_extra`

تابع `features_extra` مسئول تولید برداری از ویژگی‌های تکمیلی برای عامل RL است. این ویژگی‌ها باید بر اساس مفاهیم مدل انتخاب شوند تا یادگیری مؤثر باشد. بر اساس مدل، ویژگی‌های کلیدی زیر پیشنهاد می‌شود:

- **تاب‌آوری نسبی ($e_i^t / \bar{m}_i$):** نسبت تاب‌آوری جاری بازیکن به ظرفیت پایه می‌تواند میزان نزدیک‌شدن به خروج را نشان دهد. هرچه این نسبت بالاتر باشد، احتمال ادامه می‌تواند بیشتر باشد [L129-L137+22].
- **هزینه مؤثر (g_i^t):** هزینه کلی ادامه در این مرحله، یعنی $g_i(t, h^t; \theta_i, e_i^t) = \frac{c_i + a_i^{\{aud\}} + r_i}{m_i^t}$. این متغیر کاملاً بر تصمیم ادامه/خروج تأثیر دارد (افزایش g_i احتمال خروج را بالا می‌برد).
- **نسبت هزینه خام بر تاب‌آوری (c_i / e_i^t):** چون $g_i \propto c_i$ ، نسبت هزینه به تاب‌آوری می‌تواند شتاب فرسایش را نمایان کند.
- **ارزش خروج جاری ($X_i(t, h^t)$):** عامل باید وضعیت زمانی ارزش خروج را بداند، چون شرط ادامه در شکل $\Delta V_i = V_i^t - V_i^{t-1} \geq 0$ مقایسه می‌کند. ارزش بالاتر X_i ادامه را کم‌جذاب‌تر می‌کند [L129-L137+22].
- **تاریخچه سیگنال‌ها (حاصل جمع سیگنال‌های قوی قبلی):** برای تقریب باور عامل درباره تاب‌آوری حریف، برداری مانند تعداد مراحل "ادامه" یا توان سیگنال‌های ارسال‌شده توسط خود و حریف تا کنون کمک می‌کند. برای مثال، تعداد اقدام‌های C متوالی یا سطح کل سیگنال‌های قوی می‌تواند نشانگر نوع تقریبی باشد.
- **اعتبارات داخلی (پرداش سیگنال خرج‌شده):** جمع هزینه‌های سیگنال قبلی (متناسب با ϕ_i) که نمایانگر تعهدات قبلی و بار سیاسی است؛ در صورتی که هزینه جمعی زیاد باشد، احتمالاً X_i و $V_i^t - V_i^{t-1}$ تغییر کرده‌اند.
- **یادآوری شرط تک‌عبوری:** می‌توان فیچرهایی براساس اختلاف قدرت سیگنال‌دهی بین خود و حریف ساخت. برای مثال **میانگین هزینه سیگنال قوی به ازای هر واحد تاب‌آوری**؛ طبق مدل، این مقدار برای نوع ضعیف بسیار بزرگتر از نوع قوی است (شرط تک‌عبوری [L300-L307+41]).

مثال الگوریتم (پژوهش‌نما)

```
def features_extra(self, state, agent):
    # شامل تاریخچه و پارامترهای جاری: ورودی
    e = state['endurance'][agent]          # e_i^t
    m = state['capacity'][agent]            # m_i^t
    raw_c = state['raw_cost'][agent]        # c_i
    aud = state['audience_cost'][agent]    # a_i^{aud}
    risk = state['risk_cost'][agent]        # r_i
    X = state['exit_value'][agent]          # X_i(t, h^t)

    # هزینه مؤثر g_i
    g = (raw_c + aud + risk) / (m if m>0 else 1)
    # (مثال: e/m) نسبت تاب‌آوری
    endurance_ratio = e / m if m>0 else 0
    # نسبت هزینه به تاب‌آوری
    cost_endurance_ratio = raw_c / (e if e>0 else 1)

    # جمع سیگنال‌های قوی ارسال شده توسط عامل و حریف در گذشته
    # (را دارد C/E تاریخچه اقدامات state فرض بر این است که)
```

```

own_C_count = state['history'][agent].count('C')
opp_C_count = state['history'][1-agent].count('C')

# ویژگی‌ها در یک لیست بازمی‌گردد
features = [
    e,                # تاب‌آوری جاری
    endurance_ratio,  # نسبت تاب‌آوری
    g,                # هزینه مؤثر
    cost_endurance_ratio, # نسبت هزینه به تاب‌آوری
    own_C_count,      # تعداد ادامه‌ها توسط خود
    opp_C_count,      # تعداد ادامه‌ها توسط حریف
    X,                # ارزش خروج کنونی
]
return features

```

در این مثال، ویژگی‌ها متناسب با مولفه‌های مدل استخراج شده‌اند. برای هر فیچر می‌توان منطق نظری زیر را ذکر کرد:

- مقدار `e` و نسبت تاب‌آوری به مدیریت (`endurance_ratio`) مستقیماً میزان توان استقامت فعلی را نشان می‌دهند [L300+41-L307].
- `g` نشان‌دهنده هزینه کل ادامه است (منطبق با معادله هزینه مؤثر مدل) و هر چه بیشتر باشد، تمایل به خروج بالاتر خواهد بود [L129+22-L137].
- نسبت هزینه به تاب‌آوری (`cost_endurance_ratio`) بیانگر میزان فشاری است که به ازای هر واحد تحمل بر بازیکن وارد شده است.
- `own_C_count` و `opp_C_count` به‌عنوان سیگنال‌های ضمنی در نظر گرفته می‌شوند: تعداد دفعات ادامه (یا ارسال سیگنال قوی) از سوی یک بازیکن، حکایت از تاب‌آوری بالای او دارد. این نکته بر اساس سازوکار به‌روزرسانی باور است: ادامه مداوم یک سیگنال گران‌قیمت تلقی می‌شود [L300-L307+41] [L129-L137+22].
- نهایتاً `X` که ارزش خروج فعلی است، در تصمیم تابع ایجاد می‌شود و حضور آن به عامل امکان مقایسه مستقیم V_i^C و X_i را می‌دهد (ابطله $\backslash \text{eqref}{eq:continue}$ مدل) [L129-L137+22].

۳. الگوریتم پیشنهادی برای `shaping`

برای شکل‌دهی پاداش (`reward shaping`)، هدف استفاده از دانش نظری مدل در تغییر پاداش ابتدایی محیط به‌نحوی است که یادگیری عامل سریع‌تر شود و در عین حال تعادل‌های نظری دست‌نخورده بمانند. یک روش مرسوم «پتانسیل‌بنیاد» این کار است که با تعریف یک تابع پتانسیل $\Phi(h^t)$ روی هر تاریخچه (یا حالت) و افزودن پاداش $F(h^t, h^{t+1}) = \gamma \Phi(h^{t+1}) - \Phi(h^t)$ به پاداش اصلی، نهایتاً سیاست بهینه حفظ شود.

در این مدل، یکی از انتخاب‌های طبیعی برای تابع پتانسیل، ارزش خروج پویای جاری است:

$$X_i(h, t) = \Phi(h)$$

زیرا ماهیت تصمیمات ادامه/خروج حول مقایسه V_i^C و X_i است. بنابراین می‌توان پیشنهاد کرد:

```

def shaping(self, state, agent):
    # تعیین پتانسیل در حالت کنونی
    X_now = state['exit_value'][agent]

```

```

# تعیین حالت و پتانسیل بعد از انجام عمل
next_state = self.get_next_state(state, agent, action) # فرض وجود متد محیط
X_next = next_state['exit_value'][agent]
# فرض کنید 1 بدون تنزیل) RL پارامتر گاما
gamma = 1.0
# پاداش شکل‌دهی (متفاوت از پاداش اصلی محیط)
return gamma * X_next - X_now

```

ایده این است که اگر اقدام عامل باعث افزایش ارزش خروج شود (مثلاً فشار داخلی کمتر یا راه خروج آبرومندانه‌تر شده)، به عامل یک پاداش مثبت اضافی بدهیم و بالعکس. این نوع شکل‌دهی، عامل را ترغیب می‌کند که مسیرهایی را دنبال کند که ارزش خروج را حفظ یا افزایش می‌دهد - یعنی حفظ امکان خروج آبرومندانه. از طرف دیگر، اگر اقدام عامل ادامه بدون تغییرات بهتری در X_i منجر شود، پاداش شکل‌دهی خنثی یا منفی خواهد بود.

نکته مهم آن است که این شکل‌دهی از نوع «پتانسیل‌بنیاد» است (تحت فرض $\gamma=1$ یا هر $\gamma<1$ مناسب)، که تضمین می‌کند سیاست بهینه تغییر نکند، بلکه تنها فرآیند یادگیری سریع‌تر شود. در نوشته‌های مرجع نیز تأکید شده که شکل‌دهی باید به گونه‌ای باشد که نتیجه تعادلی بازی حفظ شود (پتانسیل‌بنیاد این خاصیت را دارد [L1-L6+42]).

۴. انتخاب پارامترها

مدل نظری و کد شبیه‌سازی چندین پارامتر کلیدی دارد. برخی در اسناد ارائه‌شده تعیین شده‌اند؛ برخی دیگر باید به‌صورت معقول انتخاب شوند. در زیر پیشنهادهایی برای مقادیر پیش‌فرض و بازه‌های منطقی آمده است:

- **تاب‌آوری اولیه (e_i^0):** معمولاً یک مقدار مثبت (مثلاً 10) در نظر گرفته می‌شود. می‌تواند برای انواع ضعیف و قوی متفاوت باشد؛ مثلاً $e_{\text{weak}}=5$ و $e_{\text{strong}}=15$. دامنه پیشنهادی: [20, 1].
- **ظرفیت پایه ($\bar{m}_i$):** میزان مدیریت هزینه اولیه. می‌تواند ثابت (مثلاً 1) یا تابعی از نوع باشد. بازه: [5, 0.5]. هرچه بیشتر باشد، تأثیر هزینه خام کمتر می‌شود (معادله eq:geff).
- **هزینه‌های فشار خام ($\{c_i, a_i^{\text{aud}}, r_i\}$):** این‌ها معمولاً وابسته به تاریخ یا متغیرهای محیط‌اند. به‌عنوان شروع، می‌توان از مقادیر ساده مانند $c_i=1$ و افزایش خطی در زمان استفاده کرد. فشار داخلی a_i^{aud} در صورت درگیری سیاسی ممکن است افزایش یابد (مثلاً بین 0 تا 10). ریسک r_i برای سادگی صفر گرفته شود یا به‌صورت تابع تصادفی کوچک.
- **ارزش برد (پیروزی):** اگر بازی با خروج حریف پایان یابد، جایزه $B_i(e_i, \theta_i)$ را دو طرف می‌گیرند. در بسیاری شبیه‌سازی‌ها می‌توان ساده فرض کرد $B_i = V > 0$ ثابت باشد (مثلاً 100) یا تابعی از تاب‌آوری. اگر در کد تعریفی وجود ندارد، یک مقدار مناسب (مثلاً 50-200) انتخاب شود.
- **فضای سیگنال (σ_i^t):** به‌طور معمول ادامه (C) یا خروج (E) دو مقدار سینگالی است. اگر امکان چند سطح قدرت سیگنال هست، می‌توان سیگنال قوی‌تر را با هزینه بالاتر تعریف کرد. به یاد داشته باشیم که $\partial \varphi / \partial e_i < 0$ (تاب‌آوری بیشتر، هزینه سیگنال کمتر) باید حفظ شود [L300-L307+41].
- **فرایند باور پیشین (μ_i^0):** معمولاً برابر فرض کردن احتمال نیمه قوی/نیمه ضعیف است. برای ساده‌سازی می‌توان $\mu_i^0(\theta_{\text{opponent}}=\text{strong})=0.5$ و $\mu_i^0(\theta_{\text{opponent}}=\text{weak})=0.5$ تنظیم کرد. اگر در کد پارامتری برای prior وجود نداشته باشد، از نرمال‌شده‌ی پارامتر نوع بازیکن دیگر استفاده شود.
- **ضریب تنزیل (γ):** اگر عامل یادگیری تقویتی از تنزیل استفاده می‌کند، معقول است برابر 1 یا نزدیک به آن باشد؛ چون بازی چندمرحله‌ای کوتاه است و تنزیل نباید بیش از حد سود بلندمدت را کم کند.

پارامترهایی که در فایل‌های موجود مشخص نشده‌اند (مانند ضرایب هزینه‌ی سیگنال $\varphi^{\text{prod}}, \varphi^{\text{commit}}$ یا ساختار دقیق X_i) باید بر اساس منطق مدل حدس زده شوند یا آزمایش گردند. مثلاً می‌توان هزینه تولید سیگنال را خطی در قدرت سیگنال و هزینه تعهد را تابعی افزایشی در قدرت و سابقه عمومی فرض کرد. در هر صورت، مقدارهای پیشنهادی بالا نقطه شروع معقولی را فراهم می‌کنند و می‌توان نتایج را با تغییر تدریجی آنها بررسی نمود.

۵. آزمون واحد و شبیه‌سازی

برای اطمینان از عملکرد صحیح توابع، چند سناریوی ساده‌ی آزمون در نظر می‌گیریم:

- **آزمون ۱ (دو نوع یکسان):** فرض کنید هر دو بازیکن قوی (تاب‌آوری زیاد و $m\$$ بالا) باشند و هزینه‌ها کم. در این حالت، هر دو میل به ادامه دارند. ویژگی‌ها باید هر دو بازیکن را دارای ارزش خروج بالاتر نشان دهند و شکل‌دهی پاداش تقریباً صفر شود (چون $X_i\$$ زیاد).
- **آزمون ۲ (یک قوی، یک ضعیف):** بازیکن ۱ تاب‌آوری قوی دارد و بازیکن ۲ ضعیف. اگر ۲ در اوایل از بازی خارج شود، بازی تمام می‌شود. در شبیه‌سازی، پس از چند مرحله ادامه ۲، باور ۱ به ضعیف بودن ۲ بالا می‌رود و ۱ پیروزی می‌گیرد. این باید در ویژگی‌ها مشخص شود (برای ۱: هزینه موثر پایین، برای ۲: بالا). شکل‌دهی باید عامل ۲ را تشویق به خروج کند وقتی $X_{2\$}$ کم است.
- **آزمون ۳ (فشار داخلی بالا):** اگر به یک بازیکن (مثلاً ۱) فشار سیاسی شدید باشد ($a_1^{aud}\$$ بالا) ولی تاب‌آوری واقعی پایین نباشد، ویژگی‌ها باید اختلاف را نشان دهند: هزینه موثر بالاتر نسبت به بازیکن ۲ و ارزش خروج پایین (کیفیت ضعیف توافق). عامل ۱ ممکن است حتی با تاب‌آوری فنی بالا، به دلیل $X_{1\$}$ پایین، در حالت مبهم قرار بگیرد. در ادامه، پاداش شکل‌دهی قطعاً ادامه را کم‌جذاب می‌کند تا خروج محتمل‌تر شود.

نمونه کدی کوتاه برای آزمون ۲:

```
# شبیه‌سازی فرضی
state = {
    'endurance': [15, 5],
    'capacity': [3, 1],
    'raw_cost': [2, 2],
    'audience_cost': [1, 5],
    'risk_cost': [0, 0],
    'exit_value': [10, 10],
    'history': [['C', 'C'], ['C', 'C']], # تا کتون هر دو ادامه داده‌اند
}
# استخراج ویژگی
features_p1 = features_extra(state, agent=0)
features_p2 = features_extra(state, agent=1)
# محاسبه پاداش شکل‌دهی اگر پ1 ادامه دهد
action = 'C'
reward_shaping = shaping(state, agent=0) # با فرض روش کار محیط
```

انتظار: در این حالت، بازیکن دوم هزینه داخلی بالا دارد ($a_2^{aud}=5\$$) و ظرفیت کم. ویژگی‌های او باید هزینه موثر (g) بزرگتر و ارزش خروج کمتر نسبت به بازیکن اول نشان دهند. پاداش شکل‌دهی برای ادامه بازیکن اول منفی یا بسیار کوچک خواهد بود (چون ارزش خروج او بالا است) و برای بازیکن دوم احتمالاً مثبت‌تر خواهد بود (چون ممکن است به زودی خروج را ترجیح دهد).

نمودار جریان داده (Flowchart)

```
flowchart LR
    S["$h^t$ حالت بازی"] --> F["features\_extra استخراج ویژگی‌ها"]
    F --> A["RL عامل"]
    A --> D["تصمیم: ادامه (C) یا خروج (E)"]
    D --> E["محیط بازی (محاسبه هزینه و سیگنال)"]
    E --> S2["$h^{t+1}$ حالت بعدی"]
```

```
S2 --> Sh[شکل‌دهی پاداش]
Sh --> R[پاداش نهایی]
R --> A
```

این نمودار چرخه یک مرحله از بازی را نشان می‌دهد: حالت فعلی به `features_extra` داده می‌شود تا بردار ویژگی تولید کند؛ عامل براساس آن تصمیم می‌گیرد؛ اجرا در محیط باعث تولید حالت بعدی و پاداش اولیه می‌شود؛ سپس تابع `shaping` پاداش را تعدیل و به عامل بازمی‌گرداند.

۶. حساسیت و پایداری

تابع‌های پیشنهادی باید نسبت به تغییرات مفروضات مدل، پایداری نسبی داشته باشند. مهم‌ترین عوامل تحت بررسی عبارت‌اند از:

- **نویز در باورها و برورزرسانی بیزی:** اگر پیش‌فرض‌های باور تغییر یا اشتباه باشند (مثلاً عامل اشتباهاً حریف را ضعیف‌تر بداند)، ممکن است ویژگی‌های مبتنی بر تاریخچه (مثل تعداد ادامه‌ها) گمراه‌کننده شوند. برای پایداری بیشتر می‌توان به جای استفاده مستقیم از تعداد، احتمال ضمنی قوی بودن (باورها) را تخمین زد و به‌عنوان ویژگی فرعی اضافه کرد. به‌علاوه، افزودن یک نویز کوچک در به‌روزرسانی می‌تواند به شناسایی بلوف‌ها کمک کند.
- **ناخالصی هزینه سیگنال (شکل غیرخطی):** اگر هزینه سیگنال به‌صورت غیرخطی (مثلاً درجه دوم) تعریف شود، شرط تک‌عبوری ممکن است دچار تغییر شود. در این حالت، می‌توان فیچر «هزینه نهایی افزایش سیگنال» را لحاظ کرد تا اثر منحنی هزینه حفظ شود. به عنوان مثال، اگر σ^2 $\varphi_i(\sigma)$ $\propto \sigma^2$ فیچر σ^2 / e^{i^2} را اضافه کنیم تا به عامل بفهماند سیگنال قوی برای ضعیف سنگین‌تر است [L300-L307+41].
- **ناهمگنی تاب‌آوری (مناطق ارزش خروج):** اگر بازیگران تاب‌آوری بسیار متفاوتی داشته باشند، آستانه خروج متنوع خواهد بود. در این شرایط، تابع `shaping` باید طوری تنظیم شود که نه کل بازی را همیشه ادامه دهد (عامل بسیار مقاوم) و نه همیشه خارج شود. تنظیم $\Phi = X_i$ باعث می‌شود عامل به حفظ پلن خروج توجه کند. همچنین در تعیین پارامترها (مثلاً دامنه $\bar{m}$ و $\bar{\rho}$) باید تنوع معقول را در نظر گرفت تا به ادغام یادگیری کمک کند.
- **تغییر در ارزش خروج (نمایه X):** اگر ارزش خروج دینامیک ناگهانی تغییر کند (مثلاً میانجیگری ناگهان ممکن شود)، شکل‌دهی پاداش می‌تواند بازی را به سرعت به سمت خروج هدایت کند. از این بابت، مدیریت حساسیت لازم است: در `shaping` ممکن است یک مقاومت نرم (مثلاً کاهش جبرانی برای جلوگیری از جهش زیاد) افزوده شود تا عامل تغییرات X را به تدریج یاد بگیرد.

در کل، پیشنهاد می‌شود پس از پیاده‌سازی اولیه، این تابع‌ها را تحت شرایط نامساختار آزمایش کرده و با افزودن نویز یا تغییر رفتار هزینه، پایداری را بررسی کنیم. اگر رفتارها به یک استراتژی پایدار و معقول در تعادل بیزی یا پشت‌آشوبگر ختم شود، نشان‌دهنده پایداری مناسب است.

۷. قطعه‌های کد پیشنهادی

در زیر کدهای فشرده‌ای برای جایگزینی در `theory.py` ارائه شده‌اند. این‌ها می‌توانند به‌عنوان نقطه شروع استفاده شوند و بسته به جزئیات محیط اصلاح گردند.

```
def features_extra(self, state, agent):
    e = state['endurance'][agent]
    m = state['capacity'][agent]
    raw_c = state['raw_cost'][agent]
    aud = state['audience_cost'][agent]
    risk = state['risk_cost'][agent]
    X = state['exit_value'][agent]
    g = (raw_c + aud + risk) / (m if m>0 else 1)
    endurance_ratio = e/(m if m>0 else 1)
    cost_end_ratio = raw_c/(e if e>0 else 1)
```

```

own_C = state['history'][agent].count('C')
opp_C = state['history'][1-agent].count('C')
return [e, endurance_ratio, g, cost_end_ratio, own_C, opp_C, X]

```

```

def shaping(self, state, agent):
    X_now = state['exit_value'][agent]
    # متد فرضی برای حالت بعدی (نیاز به پیاده‌سازی محیط)
    next_state = self.peak_next_state(agent)
    X_next = next_state['exit_value'][agent]
    gamma = 1.0
    return gamma * X_next - X_now

```

این کدها باید در فایل `theory.py` قرار گیرند و اطمینان حاصل شود که واژگان (`history`, `state`, ...) با ساختار محیط شما مطابقت دارد. دقت کنید که تابع `peak_next_state` نمونه فرضی برای شبیه‌سازی اثر عمل روی حالت است؛ در کد واقعی باید به نحوی معادل حالت جدید را محاسبه کنید.

۸. چک‌لیست ادغام و ارزیابی

- **[] نقشه متغیرها:** اطمینان از اینکه تمام ورودی‌های `state` به درستی به نمادهای مدل اختصاص یافته‌اند. (جدول مطابقت بررسی شده باشد)
- **[] عملکرد `features_extra`:** همه فیچرهای پیشنهادی محاسبه و مقادیر معقول گرفته باشند (مثلاً نسبت‌ها در بازه مناسب).
- **[] عملکرد `shaping`:** پتانسیل (تابع ارزش خروج) صحیح محاسبه شود و شکل‌دهی (احتمالاً پتانسیل‌بنیاد) بدون تغییر استراتژی بهینه اجرا شود.
- **[] مقادیر پارامترها:** پیش‌فرض‌ها مطابق پیشنهادات تعیین شده و در صورت امکان در کد مستندسازی شوند.
- **[] آزمون‌های پایه:** چندین سناریوی ساده شبیه‌سازی شده و نتایج مورد انتظار بررسی شده باشد (خروج یا ادامه معقول بازیکنان).
- **[] پایداری:** در حضور نویز یا تغییر پارامترها، رفتار عامل‌ها بیش از حد دچار آشفتگی نشود. (برای مثال، با کمی دستکاری گره‌های باور یا هزینه‌ها، نتایج قابل توجهیه باقی بمانند)
- **[] ادغام در تورنمنت:** تابع‌های جدید در `theory.py` در محیط تورنمنت جایگزین شده و از نظر عملکرد کلی (بازگشت طول عمر بازی‌ها یا درصد پیروزی) بهبود یا حداقل تغییر نامعقول ایجاد نشده باشد.

منابع: مفاهیم اساسی این طراحی بر پایه نظریه جنگ فرسایشی است که به صراحت می‌گوید بازیکنان تا زمانی ادامه می‌دهند که منافع پیشی‌گرفتن بر هزینه‌های ماندن برتری دارد [L129-L137†22]. همچنین شرط تک‌عبوری در نظریه سیگنال‌دهی بیان می‌کند ارسال سیگنال قوی برای نوع ضعیف بسیار پرهزینه است [L300-L307†41]. این ساختارها و سایر جزئیات از مقاله و دستورالعمل پروژه حاصل شده است. تمامی پیشنهادات در این گزارش نیز بر این اصول نظری استوارند.